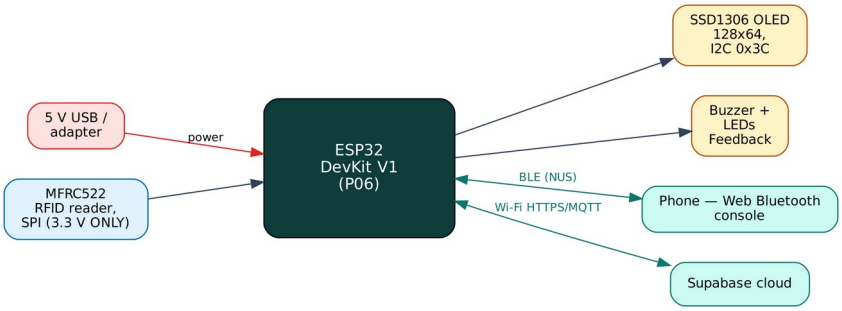
A PROJECT REPORT ON

RFID Attendance System with Offline Queue

IoT project using ESP32 with Bluetooth Low Energy and Wi-Fi (Supabase)



Project: P06 · Advanced level

Peripherals: MFRC522 RFID reader, SSD1306 OLED, Buzzer, Green/red LEDs

Submitted by: _____

Under the guidance of: _____

Department / Institution: _____

October 2026

Certificate

This is to certify that the project report entitled **“RFID Attendance System with Offline Queue”** is a bonafide record of the work carried out by _____ (Roll No. _____) in partial fulfilment of the requirements for the award of _____ in _____ during the academic year _____.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

Acknowledgement

I express my sincere gratitude to my project guide _____ for continuous guidance and support, to the Head of the Department _____, and to all faculty members and friends who helped me complete this project successfully.

Contents

Abstract 6

1. Introduction 7

1.1 Problem Statement 7
1.2 Objectives 7
1.3 Scope of the Project 7
1.4 Organisation of the Report 7

2. Existing System and Proposed System 8

2.1 Existing Systems 8
2.2 Proposed System 8
2.3 Why Both BLE and Wi-Fi? 8

3. System Design 9

3.1 Block Diagram 9
3.2 System Architecture 9
3.3 Hardware Components and Cost 9
3.4 ESP32 DevKit V1 10
3.5 Working Principle of Components 10

4. Circuit Design 11

4.1 Circuit Diagram 11
4.2 Pin Connections 11
4.3 Interfacing and Power Notes 12

5. Data Flow Diagrams 13

5.1 DFD Level 0 (Context Diagram) 13
5.2 DFD Level 1 14
5.3 Data Dictionary 14

6. Software Design and Implementation 16

6.1 Development Environment 16
6.2 Firmware Architecture 16
6.3 Working of the System 16
6.4 Flowchart 17
6.5 BLE Communication Protocol 17
6.6 Cloud Integration (Supabase) 18
6.7 Configurable Settings 18

7. Setup and Deployment Guide 19

7.1 Hardware Assembly 19
7.2 Flashing the Firmware 19
7.3 Cloud Setup (Supabase) 20
7.4 BLE App Setup (Web Bluetooth Console) 20
7.5 First-Run Checklist 21

8. Testing and Results 22

8.1 Expected Output 22
8.2 Troubleshooting 22

9. Discussion 23

9.1 Advantages 23
9.2 Limitations 23
9.3 Safety Precautions 23
9.4 Security Considerations 23
9.5 Future Scope 23

10. Conclusion **24**
References **25**
Appendix A: Project Source Code **26**
Appendix B: Shared Library Code **31**

Abstract

Paper registers and roll-calls waste class and work time and are easy to falsify. This project records attendance with 13.56 MHz RFID cards read by an MFRC522 module. Each tap is matched against a member list, toggled between IN and OUT, confirmed with an OLED message, a beep and an LED, and stored in a Supabase PostgreSQL table with a timestamp. The member list is cached in the ESP32's flash so the reader keeps working without internet; offline scans are queued in a LittleFS file and uploaded automatically when Wi-Fi returns. An administrator enrols new cards from a phone over BLE by typing a name and tapping the card. A SQL view produces the day's first-in and last-out times per member.

Keywords: ESP32, Internet of Things, Bluetooth Low Energy, Wi-Fi, Supabase, MFRC522 RFID reader, SSD1306 OLED, Buzzer, Green/red LEDs.

1. Introduction

The Internet of Things (IoT) connects physical devices — sensors and actuators — to networks so that data can be collected, analysed and acted on automatically. The ESP32 from Espressif Systems has made IoT affordable: a single ₹350 board combines a dual-core 240 MHz processor, Wi-Fi and Bluetooth Low Energy. This project uses both radios at once: BLE for direct, local interaction with a phone and Wi-Fi for cloud connectivity through Supabase, a free IoT/cloud platform.

1.1 Problem Statement

Manual attendance is slow, error-prone and allows proxy attendance. Biometric systems are costly and raise hygiene and privacy concerns, while many low-cost RFID projects lose data when the network fails and need the card list hard-coded in firmware.

1.2 Objectives

1. Identify members by RFID card UID and record IN/OUT attendance with timestamps.
2. Store attendance in a cloud database that can be queried and exported.
3. Keep working during internet outages without losing any scan.
4. Enrol and delete cards from a phone without reprogramming.
5. Give clear feedback at the reader (display, sound, LED).

1.3 Scope of the Project

Classrooms, labs, small offices and clubs with up to ~200 members per reader. Multi-reader sites can share one Supabase project. Payroll and leave management are outside the scope.

1.4 Organisation of the Report

Chapter 2 reviews existing approaches and describes the proposed system. Chapter 3 covers system design and components, Chapter 4 the circuit, Chapter 5 the data flow diagrams, Chapter 6 the software, Chapter 7 the step-by-step setup of hardware, cloud and app, Chapter 8 testing, Chapter 9 discussion and Chapter 10 the conclusion. Complete source code is given in the appendices.

2. Existing System and Proposed System

2.1 Existing Systems

Common RFID attendance projects compare UIDs against an array in the source code and send each scan straight to a spreadsheet; if Wi-Fi is down the scan is lost, and adding a member means re-flashing the board.

2.2 Proposed System

The proposed system improves on these in the following ways:

- Member list in the cloud, cached in NVS flash and refreshed every 10 minutes.
- Store-and-forward queue in LittleFS: zero data loss during outages; offline rows keep their real scan time.
- BLE administration: enrol (upsert) and delete cards from a phone.
- Automatic IN/OUT toggling and duplicate-tap protection (10 s).

2.3 Why Both BLE and Wi-Fi?

Wi-Fi gives the device internet access for remote monitoring, history and control from anywhere, but it depends on a router and an internet connection, and the device needs the Wi-Fi password before it can connect. BLE needs no infrastructure: any modern phone can connect directly within about 10-30 m. Using both gives:

- **Provisioning** — Wi-Fi credentials and cloud keys are sent from the phone over BLE and stored in flash; nothing secret is hard-coded or uploaded to GitHub.
- **Resilience** — local monitoring and control keep working when the internet or router is down.
- **Remote access** — the cloud dashboard works from anywhere when the internet is available.
- **Low cost** — no extra hardware; both radios are built into the ESP32.

The ESP32 shares one 2.4 GHz radio between Wi-Fi and Bluetooth by time-division coexistence. For this to work the Wi-Fi modem-sleep mode must stay enabled, which the firmware ensures. NimBLE, a lightweight BLE stack, is used instead of the default Bluedroid stack to leave more RAM for TLS (HTTPS/MQTTS) connections.

3. System Design

3.1 Block Diagram

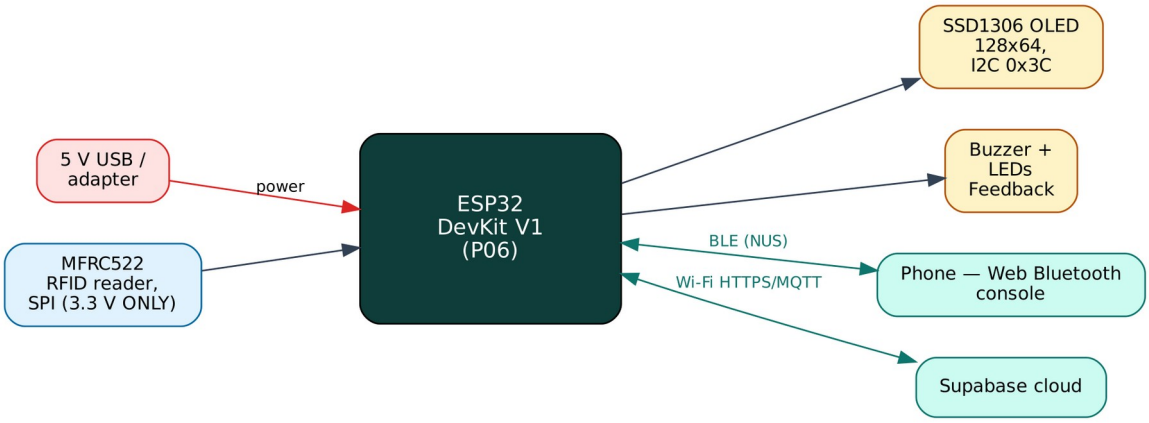


Figure 1: Block diagram of the RFID Attendance System

The ESP32 is the central controller. Input devices (blue) feed measurements to it, output devices (amber) are driven by it, and two communication links (green) connect it to the user's phone over BLE and to the Supabase cloud over Wi-Fi. The board is powered from 5 V through its USB connector or VIN pin.

3.2 System Architecture

The system follows the standard three-layer IoT architecture:

- **Perception layer** — MFRC522 RFID reader, SSD1306 OLED, Buzzer, Green/red LEDs connected to the ESP32 GPIO, ADC, I2C, SPI or UART.
- **Network layer** — BLE 4.2 GATT server (Nordic UART Service) and Wi-Fi 802.11 b/g/n with TLS-encrypted HTTPS REST to Supabase.
- **Application layer** — the Web Bluetooth console on the phone and the Supabase dashboard / database.

3.3 Hardware Components and Cost

#	Component	Qty	Specification	Cost
1	ESP32 DevKit V1 (ESP-WROOM-32, 30-pin)	1	Dual-core 240 MHz, Wi-Fi + BLE, 4 MB flash	₹350
2	Breadboard 830-point + jumper wires (M-M, M-F)	1	Prototyping and connections	₹150
3	Micro-USB data cable	1	Power + programming	₹60
4	MFRC522 RFID reader kit (with 1 card + 1 key-tag)	1	13.56 MHz, ISO 14443A, SPI, 3.3 V	₹110
5	Extra MIFARE Classic 1K cards	5	One per member	₹75
6	0.96" SSD1306 OLED 128x64 (I2C)	1	Address 0x3C	₹170
7	Active buzzer 5 V	1	Feedback	₹15
8	LEDs (green, red) + 220 Ω resistors	2	Status indication	₹5

#	Component	Qty	Specification	Cost
	Total (approx.)			**₹935**

Table 1: Bill of materials with approximate Indian prices (Robu.in / Amazon.in / local market, 2026)

> Prices are indicative and vary by seller. The breadboard, cable and ESP32 can be reused across all ten projects, so the incremental cost of this project is about ₹375.

3.4 ESP32 DevKit V1

Feature	Value
Module	ESP-WROOM-32 (ESP32-D0WD), dual-core Xtensa LX6 up to 240 MHz
Memory	520 KB SRAM, 4 MB SPI flash
Wireless	Wi-Fi 802.11 b/g/n 2.4 GHz; Bluetooth 4.2 BR/EDR + BLE
I/O	25 usable GPIO on the 30-pin board, 12-bit ADC (ADC1: GPIO32-39), 2 × DAC, PWM on any output
Interfaces	3 × UART, 2 × I2C, 3 × SPI, I2S, touch, RMT
Supply	5 V via USB/VIN, on-board 3.3 V regulator (AMS1117)
Logic level	3.3 V — inputs are NOT 5 V tolerant

Table 2: Key specifications of the ESP32 DevKit V1

3.5 Working Principle of Components

RFID (ISO 14443A)

The MFRC522 drives a 13.56 MHz field through its PCB antenna. A passive card harvests energy from this field and answers by load modulation. During anti-collision the card sends its 4- or 7-byte UID, which the firmware uses as the member ID.

SPI interface

The ESP32 talks to the MFRC522 over SPI (SCK 18, MOSI 23, MISO 19, SS 5) at up to 10 MHz; RST on GPIO4 resets the chip at start-up. The firmware checks the version register (0x91/0x92) to detect wiring faults.

LittleFS queue

LittleFS is a power-fail-safe flash file system. Each offline scan is appended as one JSON line to /queue.jsonl; after a successful upload the file is rewritten without that line.

Supabase

Two tables — members(uid, name) and attendance(uid, name, direction, scanned_at, offline) — plus an attendance_today view. The device inserts rows through the REST API using the publishable key.

4. Circuit Design

4.1 Circuit Diagram

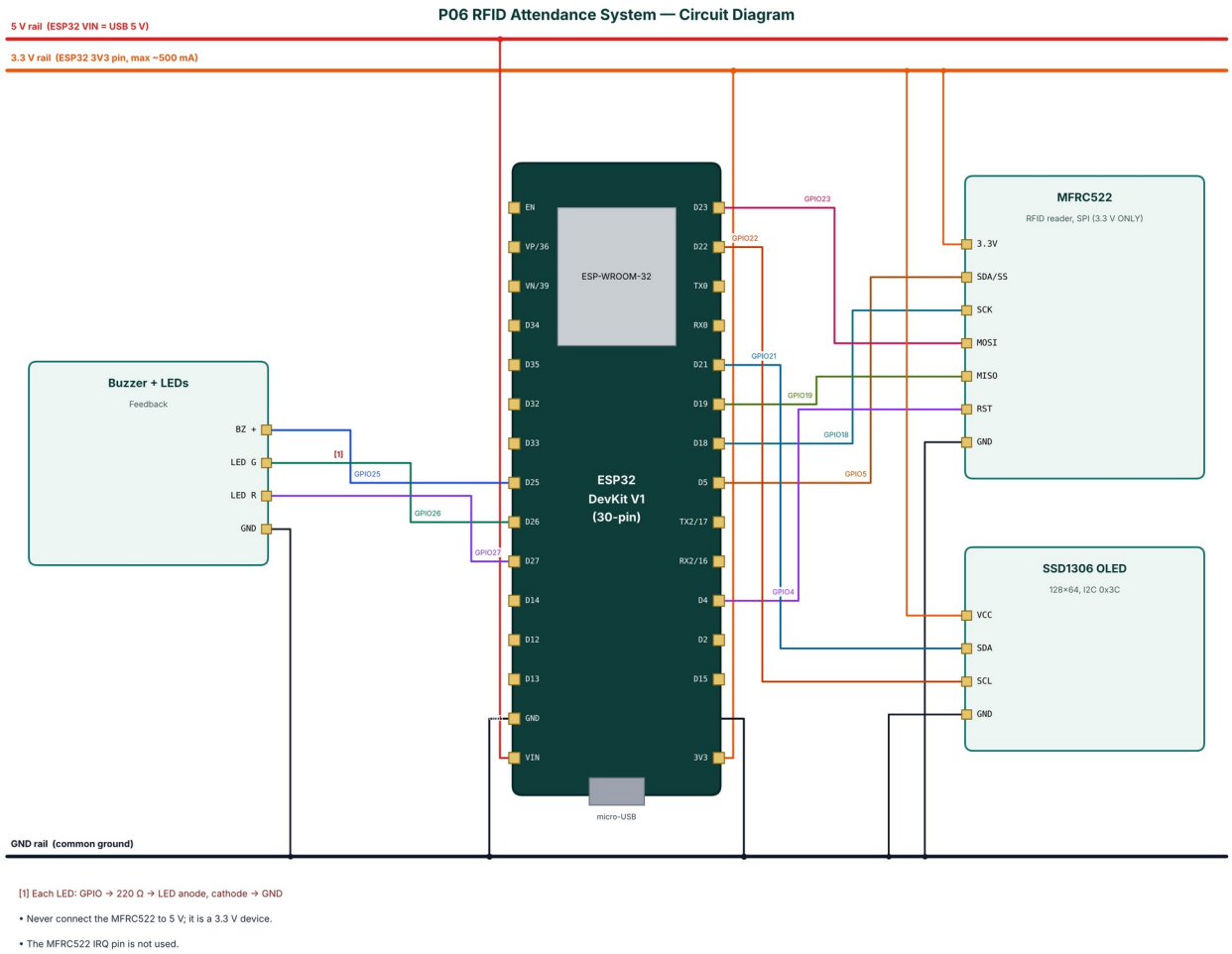


Figure 2: Circuit diagram of the RFID Attendance System

4.2 Pin Connections

Module	Module pin	ESP32 connection	Notes
MFRC522	3.3V	3.3 V (3V3 pin)	
MFRC522	SDA/SS	GPIO5	
MFRC522	SCK	GPIO18	
MFRC522	MOSI	GPIO23	
MFRC522	MISO	GPIO19	
MFRC522	RST	GPIO4	
MFRC522	GND	GND	
SSD1306 OLED	VCC	3.3 V (3V3 pin)	
SSD1306 OLED	SDA	GPIO21	
SSD1306 OLED	SCL	GPIO22	
SSD1306 OLED	GND	GND	

Module	Module pin	ESP32 connection	Notes
Buzzer + LEDs	BZ +	GPIO25	
Buzzer + LEDs	LED G	GPIO26	Each LED: GPIO -> 220 Ω -> LED anode, cathode -> GND
Buzzer + LEDs	LED R	GPIO27	
Buzzer + LEDs	GND	GND	

Table 3: Pin connection table

4.3 Interfacing and Power Notes

- Never connect the MFRC522 to 5 V; it is a 3.3 V device.
- The MFRC522 IRQ pin is not used.
- Only ADC1 pins (GPIO32-39) are used for analog inputs, because ADC2 cannot be read while Wi-Fi is active.
- Strapping pins GPIO0, 2, 12 and 15 are avoided for inputs so the board always boots normally.
- All grounds are common. 5 V signals are reduced to 3.3 V with resistor dividers before reaching the ESP32.

5. Data Flow Diagrams

Data flow diagrams (DFDs) show how data moves through the system without showing control logic or timing. External entities are drawn as rectangles, processes as numbered circles, data stores as open boxes and data flows as labelled arrows.

5.1 DFD Level 0 (Context Diagram)

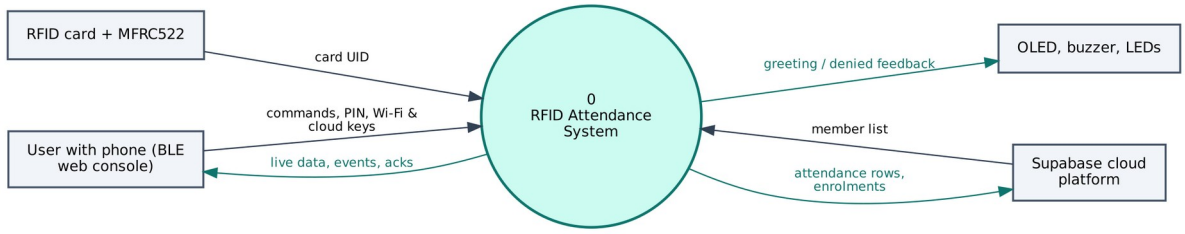


Figure 3: DFD Level 0 — context diagram

At level 0 the whole RFID Attendance System is a single process. It receives card UID from the RFID card + MFRC522, commands and configuration from the user's phone, and commands from Supabase. It sends greeting / denied feedback to the OLED, buzzer, LEDs, live data to the phone and attendance rows, enrolments to Supabase.

5.2 DFD Level 1

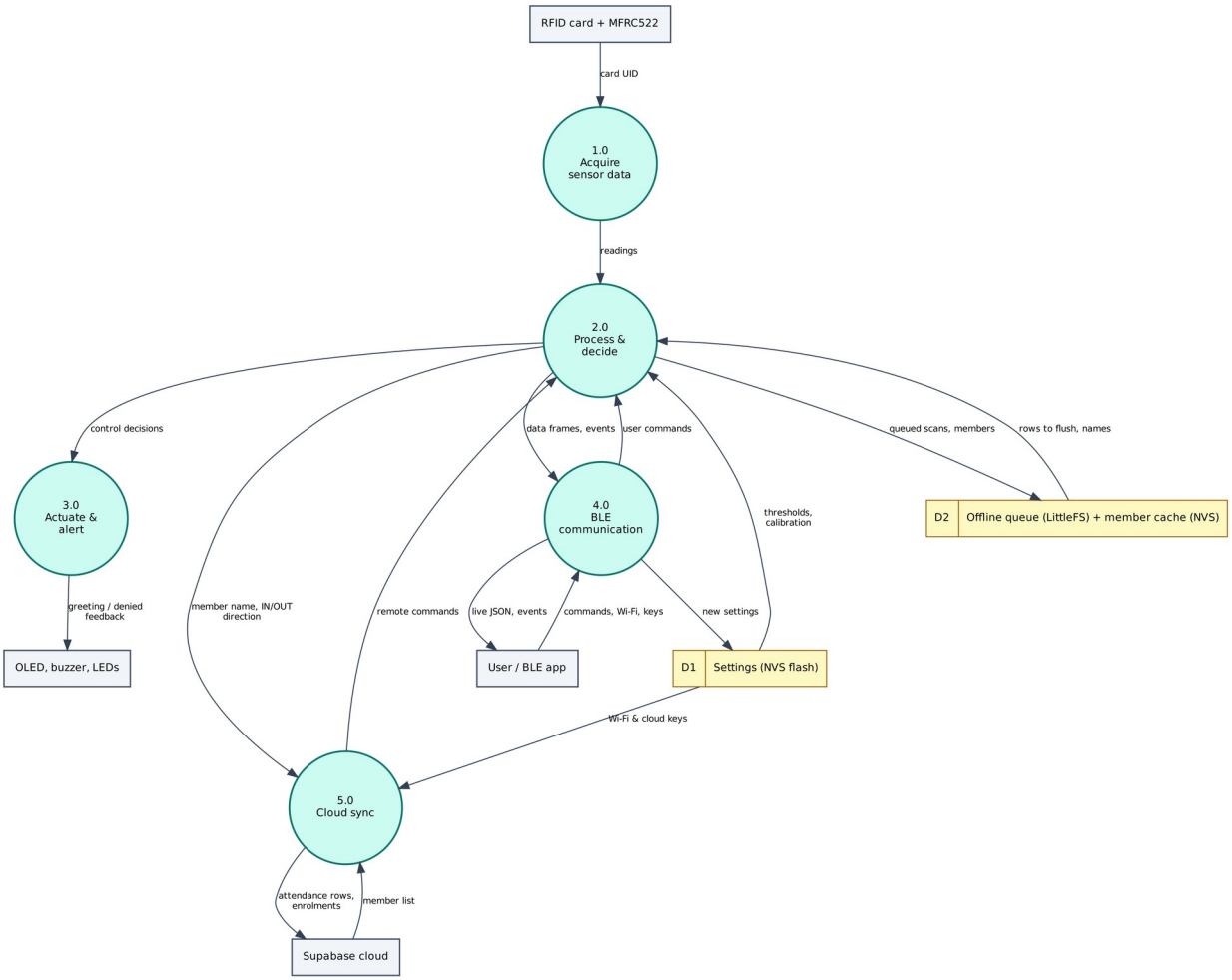


Figure 4: DFD Level 1 — main processes and data stores

Process / store	Description
1.0 Acquire sensor data	Reads card UID from the RFID card + MFRC522; filters/averages the raw values.
2.0 Process & decide	Computes member name, IN/OUT direction; applies thresholds and rules; generates events.
3.0 Actuate & alert	Drives the OLED, buzzer, LEDs (greeting / denied feedback).
4.0 BLE communication	Sends JSON frames and events to the phone; receives and authenticates commands.
5.0 Cloud sync	Sends attendance rows, enrolments to Supabase; receives member list.
D1 Settings (NVS)	Wi-Fi credentials, cloud keys, PIN, thresholds — survive reboots.
D2 Offline queue (LittleFS) + member cache (NVS)	Stores queued scans, members; returns rows to flush, names.

Table 4: Description of level-1 processes and data stores

5.3 Data Dictionary

Field	Meaning	Unit
today	Scans since boot	count
queued	Rows waiting to upload	count

Field	Meaning	Unit
enrol	Enrol mode active	bool
wifi	Wi-Fi connected	bool
type=scan	Per tap: uid, name, dir, cloud	line

Table 5: BLE data frame fields

Cloud object	Content / usage
members	uid (PK, upper-case hex), name, role
attendance	uid, name, direction IN/OUT, device, offline, scanned_at
attendance_today (view)	first_in, last_out and scans per member for today (IST)

Table 6: Supabase data objects

6. Software Design and Implementation

6.1 Development Environment

Item	Details
IDE / build	Arduino IDE 2.x or arduino-cli; GitHub Actions builds firmware in the cloud
Board package	esp32 by Espressif Systems 3.x (tested with 3.3.12)
Board settings	ESP32 Dev Module, Partition Scheme: Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)
Common libraries	NimBLE-Arduino 2.x, ArduinoJson 7.x, PubSubClient 2.8
Project libraries	MFRC522 (GithubCommunity / miguelbalboa), Adafruit SSD1306, Adafruit GFX Library
Phone app	Web Bluetooth console (Chrome on Android / desktop, Bluefy on iPhone); nRF Connect as alternative
Cloud	Supabase (free tier)

Table 7: Software tools

> The default 1.2 MB app partition is too small for BLE + Wi-Fi + TLS. Select **Minimal SPIFFS (1.9MB APP)** in Tools → Partition Scheme, otherwise the upload fails with 'Sketch too big'.

6.2 Firmware Architecture

The firmware is split into three layers so that the ten projects share tested code:

- **IoTCore.h** (shared) — Wi-Fi station with auto-reconnect, NimBLE GATT server with the Nordic UART Service, line-based command parser shared by BLE and USB serial, PIN authentication, settings in NVS (Preferences), NTP time in IST, JSON helpers.
- **Cloud.h** (shared) — ThingSpeak HTTPS client, Adafruit IO MQTT/TLS client with a 28 messages/minute rate limiter, Supabase PostgREST client (insert, update, select).
- **P06_RFID_Attendance_System.ino** — sensor drivers, algorithms, automation rules and the project's own commands.

The main loop is non-blocking: every task is scheduled with millis() timers, BLE writes are queued from the BLE callback into a FreeRTOS queue and executed in loop(), so the BLE stack never waits for slow operations such as HTTPS requests.

6.3 Working of the System

On boot the firmware mounts LittleFS, counts queued rows, initialises the reader and OLED and, once Wi-Fi is up, downloads the member list (uid, name) from Supabase into the 'cards' NVS namespace.

When a card is presented the UID is read as upper-case hex. A repeated tap within 10 s is ignored. If the UID is unknown the red LED and a long beep signal 'ACCESS DENIED' and an 'unknown_card' event is sent over BLE. Otherwise the IN/OUT flag for that UID in NVS is toggled, the OLED greets the member and a row is built with uid, name, direction, device and (if NTP time is known) scanned_at.

If Wi-Fi is up and the queue is empty the row is inserted directly; otherwise it is appended to the queue with offline=true. Every 15 s the queue is flushed row by row; the first failure stops the flush so order is kept. In enrolment mode (enroll:<name>) the next card is stored locally and upserted into members with 'Prefer: resolution=merge-duplicates'.

6.4 Flowchart

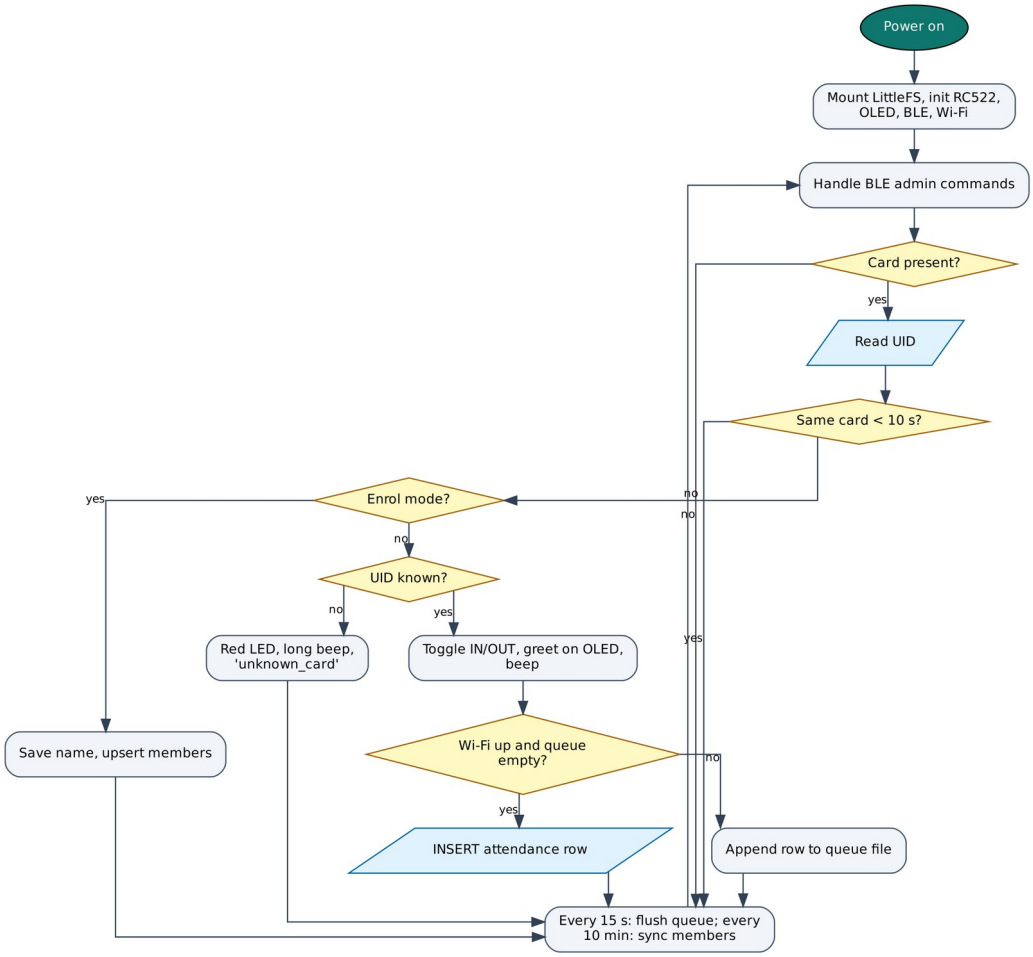


Figure 5: Firmware flowchart

6.5 BLE Communication Protocol

The device is a BLE peripheral named **IOT-P06-XXXX** (XXXX = last bytes of the MAC address). It exposes the Nordic UART Service, which most BLE apps understand:

Item	UUID	Properties
Service (NUS)	6E400001-B5A3-F393-E0A9-E50E24DCCA9E	Primary
RX — phone → ESP32	6E400002-B5A3-F393-E0A9-E50E24DCCA9E	Write, Write without response
TX — ESP32 → phone	6E400003-B5A3-F393-E0A9-E50E24DCCA9E	Notify, Read

Table 8: GATT service and characteristics

Commands are plain text lines (`command:argument`). Replies are one JSON object per line ending in a newline; long lines are split into MTU-sized notifications and re-joined by the app. Commands that change anything require `auth:<PIN>` first on each connection (default PIN 1234, change it with `set:pin=XXXX`).

Command	PIN needed	Action
auth:<PIN>	—	Unlock for this connection
status	No	Status JSON: Wi-Fi, IP, RSSI, heap, uptime, time
wifi:<SSID>,<password>	Yes	Save Wi-Fi credentials and connect

Command	PIN needed	Action
set:<key>=<value>	Yes	Save a setting (cloud keys, thresholds)
get:<key>	Yes	Read a setting (secrets masked)
reboot / forget	Yes	Restart / erase all settings
enroll:<name>	Yes	Next card tapped (within 30 s) is enrolled as <name>
delete:<UID>	Yes	Remove a card locally and from the cloud
cancel	Yes	Leave enrol mode
syncnow	Yes	Refresh members and flush the queue now

Table 9: BLE commands (common commands first, then project commands)

Example data frame sent by the device:

```
{"type":"scan","uid":"A1B2C3D4","name":"Rohit","dir":"IN","cloud":true}
```

6.6 Cloud Integration (Supabase)

Supabase exposes every Postgres table through a REST API (PostgREST). The device sends HTTPS requests to `https://<project>.supabase.co/rest/v1/<table>` with the `apikey` and `Authorization: Bearer` headers: POST to insert rows, PATCH with a filter such as `?id=eq.1` to update and GET with `?select=` to read. Row Level Security policies (in cloud/supabase_schema.sql) allow the device key to do only what the project needs.

Object	Content
members	uid (PK, upper-case hex), name, role
attendance	uid, name, direction IN/OUT, device, offline, scanned_at
attendance_today (view)	first_in, last_out and scans per member for today (IST)

Table 10: Supabase configuration

6.7 Configurable Settings

Key	Default	Meaning
sburl / sbkey	—	Supabase project URL and publishable key
ssid / pass	—	Wi-Fi network (set with the wifi: command)
pin	1234	BLE PIN

Table 11: Settings stored in flash (change with set:key=value)

7. Setup and Deployment Guide

7.1 Hardware Assembly

1. Place the ESP32 DevKit across the centre gap of the breadboard so both pin rows are free.
2. Connect the ESP32 3V3 pin to one breadboard power rail (3.3 V), VIN to the other rail (5 V) and both GND pins to the ground rails.
3. Wire each module exactly as in the pin connection table (section 4.2), including the resistor dividers marked with [n] in the circuit diagram.
4. Check every connection against the circuit diagram before powering on. Make sure no 5 V signal reaches an ESP32 pin directly.
5. Connect the USB cable to a computer (for flashing) or a 5 V adapter (for normal use).

7.2 Flashing the Firmware

Option A — Arduino IDE

1. Install Arduino IDE 2.x from arduino.cc.
2. File → Preferences → Additional boards manager URLs: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`.
3. Tools → Board → Boards Manager: install **esp32 by Espressif Systems** (version 3.x).
4. Sketch → Include Library → Manage Libraries: install **NimBLE-Arduino** (2.x), **ArduinoJson** (7.x), **PubSubClient**, **MFRC522 (GithubCommunity / miguelbalboa)**, **Adafruit SSD1306**, **Adafruit GFX Library**.
5. Download the repository and open the folder **P06_RFID_Attendance_System** (it already contains IoTCore.h, Cloud.h and config.h).
6. Tools: Board **ESP32 Dev Module**, Partition Scheme **Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)**, Upload Speed 921600, and the correct COM port.
7. Click Upload. If it stays at 'Connecting...', hold the BOOT button on the ESP32 until upload starts.
8. Open Serial Monitor at 115200 baud and type `status` to check the device.

Option B — no software installation (GitHub Actions + browser flasher)

1. Open the repository https://github.com/rohitkukreja705/esp32-iot-projects → **Actions** → latest **Build firmware** run.
2. Download the artifact **firmware-P06_RFID_Attendance_System** and unzip it to get `P06_RFID_Attendance_System.ino.merged.bin`.
3. Open **https://espressif.github.io/esptool-js/** in Chrome or Edge on a computer, click **Connect** and select the ESP32's serial port.
4. Set Flash Address **0x0**, choose the merged .bin file and click **Program**. Press the EN (reset) button when done.

7.3 Cloud Setup (Supabase)

1. Go to **supabase.com**, sign in (GitHub sign-in works) and click **New project**. Choose region **Mumbai (ap-south-1)** for the lowest latency and set a database password.
2. Open **SQL Editor → New query**, paste the whole file `cloud/supabase_schema.sql` from the repository and click **Run**. It creates the tables, views and Row Level Security policies for all four Supabase projects (P05, P06, P09, P10), so one Supabase project can serve all of them.
3. Click **Connect** at the top of the dashboard (or **Project Settings → API Keys**) and copy the **Project URL** (https://xxxx.supabase.co) and the **publishable key** (`sb_publishable_…`; the legacy `anon` key also works).
4. In the BLE web console open **Setup**, unlock and save them in `sburl` and `sbkey`.
5. Open **Table Editor** to watch rows arrive. Use the **SQL Editor** for reports, e.g. the views created by the schema, and **Export to CSV** for spreadsheets.
6. Free tier: 500 MB database and 2 active projects. Free projects are **paused after one week of inactivity** — open the dashboard and click **Restore** if that happens.

Field / feed / table	Content
members	uid (PK, upper-case hex), name, role
attendance	uid, name, direction IN/OUT, device, offline, scanned_at
attendance_today (view)	first_in, last_out and scans per member for today (IST)

Table 12: Items to create in Supabase

7.4 BLE App Setup (Web Bluetooth Console)

No app installation is needed. The console is a web page hosted free on GitHub Pages; Chrome talks to the ESP32 directly through the Web Bluetooth API.

1. On an Android phone turn on **Bluetooth** and **Location** (Android requires location for BLE scanning).
2. Open **https://rohitkukreja705.github.io/esp32-iot-projects/?p=P06** in **Chrome**. (On iPhone use the free **Bluefy** browser; on a laptop use Chrome or Edge.)
3. Tap **Connect** and choose **IOT-P06-XXXX** in the pop-up list. The dot turns green and the Live tab starts updating.
4. Open **Setup**, enter the PIN (default **1234**) and tap **Unlock**.
5. Enter your 2.4 GHz Wi-Fi name and password and tap **Save Wi-Fi**. After a few seconds the status line shows the IP address.
6. Enter the Supabase keys from section 7.3 and tap **Save** next to each.
7. Use the **Control** tab for commands and the **Log** tab to see every event and acknowledgement.

Alternative: nRF Connect app

1. Install **nRF Connect for Mobile** (Nordic Semiconductor, free) and scan.
2. Connect to **IOT-P06-XXXX** and open **Nordic UART Service**.
3. Enable notifications on the **TX** characteristic (6E400003…).
4. Write UTF-8 text to the **RX** characteristic (6E400002…), e.g. `auth:1234`, then `status`.

> To host the console yourself: in the GitHub repository open Settings → Pages, choose 'Deploy from a branch', branch main, folder /docs. Web Bluetooth only works on HTTPS pages, which GitHub Pages provides.

7.5 First-Run Checklist

- Serial Monitor shows '[BLE] advertising as IOT-P06-XXXX'.
- Status in the console shows wifi: true and an IP address.
- Data appears in Supabase within a minute.
- All sensor values in the Live tab are plausible (no '—').

8. Testing and Results

The following test cases verify each function. Record the observed result and mark Pass/Fail while testing your build.

#	Test	Procedure	Expected result	Observed
T1	Reader detection	Open Serial Monitor at boot	'[RC522] version 0x92 OK' (or 0x91)	
T2	Enrolment	Unlock, send enroll:Rohit, tap a card	OLED 'ENROLLED'; row appears in members table	
T3	IN / OUT	Tap the card, wait 11 s, tap again	Rows with direction IN then OUT in attendance	
T4	Unknown card	Tap a card that is not enrolled	'ACCESS DENIED', red LED, long beep	
T5	Duplicate tap	Tap the same card twice within 5 s	Only one row recorded	
T6	Offline queue	Disconnect Wi-Fi, tap 3 times, reconnect	OLED shows Q:3, then rows uploaded with offline=true	
T7	Daily report	Run 'select * from attendance_today' in SQL editor	One line per member with first IN and last OUT	

Table 13: Test cases

8.1 Expected Output

A typical data frame received in the console's Log tab looks like this (values will differ):

```
{"type":"scan","uid":"A1B2C3D4","name":"Rohit","dir":"IN","cloud":true}
```

Serial Monitor (115200 baud) prints the same lines along with connection messages such as `[WiFi] connected`, `[BLE] connected` and the HTTP or MQTT result of each cloud upload, which helps in troubleshooting.

8.2 Troubleshooting

Symptom	Likely cause and fix
Device not in the Connect list	Bluetooth/Location off on the phone, or the page is not opened in Chrome over HTTPS.
'Sketch too big' on upload	Select Partition Scheme 'Minimal SPIFFS (1.9MB APP with OTA)'.
Wi-Fi never connects	ESP32 supports only 2.4 GHz networks; check the SSID/password (case-sensitive).
Settings refused	Send auth:<PIN> first; the unlock is per BLE connection.
Sensor shows null / —	Check wiring and supply voltage; see the Serial Monitor for 'not found' messages.
Board resets randomly	Weak USB supply or servo/relay current; use a 5 V 2 A adapter and add a 470 µF capacitor.
No data in Supabase	Wrong URL/key, schema not run, or project paused after inactivity.

Table 14: Troubleshooting guide

9. Discussion

9.1 Advantages

- No data loss during network outages (store-and-forward).
- Cards managed from a phone; no reprogramming.
- Standard SQL database: easy reports, CSV export, integration.
- Fast (< 1 s per tap) and contactless.

9.2 Limitations

- Cards can be lent to others (proxy) — RFID identifies the card, not the person.
- MIFARE Classic UIDs can be cloned with special tools; use for low-security attendance only.
- Scans made before NTP sync get the upload time instead of the real time.
- The publishable key with permissive RLS is suitable for trusted environments only.

9.3 Safety Precautions

- Store only the minimum personal data (name, card UID) and inform members how it is used.
- Restrict Supabase table access with RLS and rotate the key if a device is lost.

9.4 Security Considerations

- Secrets (Wi-Fi password, cloud keys) are stored in the ESP32's NVS flash, not in source code.
- BLE commands that change the device require a PIN; change the default PIN after setup.
- Cloud traffic is TLS-encrypted. Certificate verification is disabled (setInsecure) for simplicity; for production, pin the server's root certificate with setCACert().
- Use a dedicated cloud account/project for the device and rotate keys if the device is lost.

9.5 Future Scope

- Add a fingerprint sensor (R307) or face check for two-factor attendance.
- Web dashboard with Supabase Auth for teachers/managers and CSV export.
- Real-time push notifications to parents/managers using Supabase Realtime.
- Battery backup and RTC module (DS3231) for accurate offline timestamps.

10. Conclusion

The RFID Attendance System replaces paper registers with a fast, contactless and reliable system. Caching the member list and queuing scans in flash makes it robust against internet outages, while Supabase provides a real relational database for reports. Phone-based enrolment over BLE keeps administration simple.

All design goals listed in Chapter 1 are met with low-cost parts available in the Indian market, free cloud services and open-source software. The shared firmware core and the single Web Bluetooth console make the project easy to reproduce and extend.

References

1. NXP Semiconductors, MFRC522 Standard performance MIFARE and NTAG frontend datasheet.
2. M. Balboa, Arduino RFID library for MFRC522, GitHub (miguelbalboa/rfid).
3. ISO/IEC 14443-3: Identification cards — Proximity cards — Initialization and anticollision.
4. Supabase documentation: Upsert with PostgREST (Prefer: resolution=merge-duplicates).
5. Espressif Systems, ESP32 Series Datasheet and ESP32 Technical Reference Manual, espressif.com.
6. Espressif Systems, Arduino-ESP32 documentation, docs.espressif.com/projects/arduino-esp32.
7. H2zero, NimBLE-Arduino library documentation, github.com/h2zero/NimBLE-Arduino.
8. Bluetooth SIG, Bluetooth Core Specification 4.2 — Generic Attribute Profile (GATT).
9. W3C Web Bluetooth Community Group, Web Bluetooth specification, webbluetoothcg.github.io/web-bluetooth.
10. B. Blanchon, ArduinoJson 7 documentation, arduinojson.org.
11. Supabase Inc., Supabase documentation — REST API and Row Level Security, supabase.com/docs.

Appendix A: Project Source Code

The complete code is in the repository folder **P06_RFID_Attendance_System** (https://github.com/rohitkukreja705/esp32-iot-projects).

config.h

```
#pragma once
// ===== P06 RFID Attendance System — configuration =====
#define WIFI_SSID     ""
#define WIFI_PASS     ""
#define DEVICE_PIN    "1234"
#define SUPABASE_URL  ""
#define SUPABASE_KEY  ""

// ----- pins -----
// MFRC522 (3.3 V ONLY!): SDA/SS 5, SCK 18, MOSI 23, MISO 19, RST 4
#define PIN_RC522_SS  5
#define PIN_RC522_RST 4
#define PIN_BUZZER    25
#define PIN_LED_OK    26     // green LED + 220R
#define PIN_LED_ERR   27     // red LED + 220R
// SSD1306 OLED: SDA 21, SCL 22
#define OLED_ADDR     0x3C

// ----- behaviour -----
#define REPEAT_BLOCK_MS   10000   // ignore the same card for 10 s
#define MEMBER_SYNC_MS    600000  // refresh member list from Supabase every 10 min
#define MAX_MEMBERS       200
```

P06_RFID_Attendance_System.ino

```
/*
 * P06 — RFID Attendance System with Offline Queue
 * ================================================
 * Peripherals: MFRC522 RFID reader (13.56 MHz cards/tags), SSD1306 OLED,
 *              buzzer, green/red status LEDs
 * Wi-Fi  : Supabase — "members" table (who owns which card) and
 *          "attendance" table (IN/OUT log with timestamps)
 * BLE    : admin enrolment (enroll:<name>, then tap the card), delete cards,
 *          live scan feed on the phone
 * Extras : works without internet — members are cached in flash and scans
 *          are queued in LittleFS, then flushed when Wi-Fi returns;
 *          automatic IN/OUT toggling; duplicate-tap protection
 */
#include "config.h"
#include "IoTCore.h"
#include "Cloud.h"
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <LittleFS.h>

MFRC522 rfid(PIN_RC522_SS, PIN_RC522_RST);
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
Preferences cards;     // uid -> name   (namespace "cards")
Preferences inout;     // uid -> "1" when currently IN (namespace "inout")
bool oledOK = false, fsOK = false;
```

```
String enrollName = "";            // non-empty while waiting for a card to enrol
uint32_t enrollUntil = 0;
String lastUid; uint32_t lastUidAt = 0;
uint32_t lastMemberSync = 0, lastFlush = 0, screenUntil = 0;
int scansToday = 0, queued = 0;

String uidHex(const MFRC522::Uid& u) {
  String s;
  for (byte i = 0; i < u.size; i++) { if (u.uidByte[i] < 0x10) s += "0"; s += String(u.uidByte[i], HEX);
}
  s.toUpperCase();
  return s;
}

void beep(int times, int ms) {
  for (int i = 0; i < times; i++) { digitalWrite(PIN_BUZZER, HIGH); delay(ms); digitalWrite(PIN_BUZZER,
LOW); delay(80); }
}

void show(const String& l1, const String& l2, const String& l3, uint32_t holdMs = 3000) {
  if (!oledOK) return;
  oled.clearDisplay(); oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(1); oled.setCursor(0, 0); oled.print(l1);
  oled.setTextSize(2); oled.setCursor(0, 16); oled.print(l2.substring(0, 10));
  oled.setTextSize(1); oled.setCursor(0, 44); oled.print(l3);
  oled.setCursor(0, 56);
  oled.printf("%s %s Q:%d", iot::wifiOK() ? "WiFi" : "----", iot::bleConnected() ? "BLE" : "---",
queued);
  oled.display();
  screenUntil = holdMs ? millis() + holdMs : 0;
}

void idleScreen() {
  String t = iot::timestamp();
  show("RFID ATTENDANCE", "Tap card", t.length() > 16 ? t.substring(11, 16) + "  scans:" +
String(scansToday) : "time not synced", 0);
}

// ------------------------------------------------------------ offline queue
void queueRecord(const String& json) {
  if (!fsOK) return;
  File f = LittleFS.open("/queue.jsonl", FILE_APPEND);
  if (f) { f.println(json); f.close(); queued++; }
}

void countQueue() {
  queued = 0;
  if (!fsOK || !LittleFS.exists("/queue.jsonl")) return;
  File f = LittleFS.open("/queue.jsonl", FILE_READ);
  while (f && f.available()) { if (f.readStringUntil('\n').length() > 2) queued++; }
  if (f) f.close();
}

void flushQueue() {
  if (!fsOK || !iot::wifiOK() || queued == 0) return;
  File f = LittleFS.open("/queue.jsonl", FILE_READ);
  if (!f) return;
  // Rows are posted one by one (offline rows may differ in their keys).
  // Stop at the first failure and keep that row and the rest for later.
  String rest; int sent = 0; bool failed = false;
  while (f.available()) {
    String line = f.readStringUntil('\n'); line.trim();
    if (line.length() < 3) continue;
```

```
    if (!failed && sent < 20 && cloud::supabaseInsert("attendance", line)) { sent++; continue; }
    failed = true;
    rest += line + "\n";
  }
  f.close();
  File w = LittleFS.open("/queue.jsonl", FILE_WRITE);
  if (w) { w.print(rest); w.close(); }
  countQueue();
  if (sent) iot::sendEvent("sync", "Uploaded " + String(sent) + " offline scans");
}

// ------------------------------------------------------------ members
void syncMembers() {
  String out;
  if (!cloud::supabaseSelect("members", "select=uid,name&limit=" + String(MAX_MEMBERS), out)) return;
  JsonDocument d;
  if (deserializeJson(d, out)) return;
  int n = 0;
  for (JsonObject m : d.as<JsonArray>()) {
    String uid = m["uid"] | ""; String name = m["name"] | "";
    if (uid.length() && uid.length() <= 15 && cards.getString(uid.c_str(), "") != name)
cards.putString(uid.c_str(), name);
    n++;
  }
  Serial.printf("[MEMBERS] %d members synced\n", n);
}

void enrolCard(const String& uid) {
  cards.putString(uid.c_str(), enrollName);
  JsonDocument d; d["uid"] = uid; d["name"] = enrollName;
  String s; serializeJson(d, s);
  // upsert so re-enrolling the same card renames it
  int c = cloud::supabaseRequest("POST", "/rest/v1/members", s, nullptr,
                                 "resolution=merge-duplicates,return=minimal");
  iot::sendEvent("enrolled", enrollName + " -> " + uid + (c >= 200 && c < 300 ? " (cloud ok)" : " (saved
locally)"));
  show("ENROLLED", enrollName, uid);
  beep(2, 80);
  enrollName = "";
}

void handleScan(const String& uid) {
  uint32_t now = millis();
  if (uid == lastUid && now - lastUidAt < REPEAT_BLOCK_MS) return;   // double tap
  lastUid = uid; lastUidAt = now;

  if (enrollName.length()) { enrolCard(uid); return; }

  String name = cards.getString(uid.c_str(), "");
  if (name.isEmpty()) {
    digitalWrite(PIN_LED_ERR, HIGH); beep(1, 600); digitalWrite(PIN_LED_ERR, LOW);
    show("ACCESS DENIED", "Unknown", uid);
    iot::sendEvent("unknown_card", uid);
    return;
  }
  bool wasIn = inout.getString(uid.c_str(), "0") == "1";
  const char* dir = wasIn ? "OUT" : "IN";
  inout.putString(uid.c_str(), wasIn ? "0" : "1");
  scansToday++;

  digitalWrite(PIN_LED_OK, HIGH); beep(1, 120); digitalWrite(PIN_LED_OK, LOW);
  String t = iot::timestamp();
```

```
  show(String(dir) == "IN" ? "WELCOME" : "GOODBYE", name, String(dir) + "  " + (t.length() > 16 ?
t.substring(11, 19) : ""));

  JsonDocument d;
  d["uid"] = uid; d["name"] = name; d["direction"] = dir; d["device"] = iot::deviceName;
  if (t.startsWith("20")) d["scanned_at"] = t;   // keep the real scan time for offline rows
  String row; serializeJson(d, row);

  bool sent = iot::wifiOK() && queued == 0 && cloud::supabaseInsert("attendance", row);
  if (!sent) { d["offline"] = true; row = ""; serializeJson(d, row); queueRecord(row); }

  JsonDocument e;
  e["type"] = "scan"; e["uid"] = uid; e["name"] = name; e["dir"] = dir; e["cloud"] = sent;
  String s; serializeJson(e, s); iot::emit(s);
}

// Commands: enroll:<name>  delete:<UID>  list  cancel
bool onCommand(const String& cmd, const String& arg, bool authed) {
  if (cmd == "setting") return true;
  if (cmd == "enroll" || cmd == "delete" || cmd == "cancel" || cmd == "list" || cmd == "syncnow") {
    if (!authed) { iot::emitAck(cmd, false, "send auth:<PIN> first"); return true; }
    if (cmd == "enroll") {
      if (arg.isEmpty()) { iot::emitAck(cmd, false, "use enroll:<name>"); return true; }
      enrollName = arg; enrollUntil = millis() + 30000;
      show("ENROL MODE", "Tap card", "for " + arg, 30000);
      iot::emitAck(cmd, true, "tap the card within 30 s");
    } else if (cmd == "delete") {
      cards.remove(arg.c_str()); inout.remove(arg.c_str());
      cloud::supabaseRequest("DELETE", "/rest/v1/members?uid=eq." + arg, "");
      iot::emitAck(cmd, true, arg + " removed");
    } else if (cmd == "cancel") {
      enrollName = ""; iot::emitAck(cmd, true);
    } else if (cmd == "syncnow") {
      syncMembers(); flushQueue(); iot::emitAck(cmd, true, "queued=" + String(queued));
    } else {
      iot::emitAck(cmd, true, "members cached: " + String(cards.freeEntries()) + " free NVS entries");
    }
    return true;
  }
  return false;
}

void sendHeartbeat() {
  JsonDocument d;
  d["today"] = scansToday; d["queued"] = queued; d["enrol"] = enrollName.length() > 0;
  d["wifi"] = iot::wifiOK();
  iot::sendData(d);
}

void setup() {
  pinMode(PIN_BUZZER, OUTPUT); pinMode(PIN_LED_OK, OUTPUT); pinMode(PIN_LED_ERR, OUTPUT);
  iot::begin("P06", onCommand);
  cards.begin("cards", false);
  inout.begin("inout", false);
  fsOK = LittleFS.begin(true);              // format on first use
  countQueue();
  SPI.begin(18, 19, 23, PIN_RC522_SS);
  rfid.PCD_Init();
  byte v = rfid.PCD_ReadRegister(MFRC522::VersionReg);
  Serial.printf("[RC522] version 0x%02X %s\n", v, (v == 0x91 || v == 0x92 || v == 0x88) ? "OK" : "(check
wiring)");
  Wire.begin(21, 22);
  oledOK = oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
```

```
  idleScreen();
}

void loop() {
  iot::loop();
  uint32_t now = millis();

  if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
    handleScan(uidHex(rfid.uid));
    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();
  }
  if (enrollName.length() && now > enrollUntil) { enrollName = ""; iot::sendEvent("enrol", "timed out");
}
  if (screenUntil && now > screenUntil) { screenUntil = 0; idleScreen(); }

  if (iot::wifiOK() && (lastMemberSync == 0 || now - lastMemberSync > MEMBER_SYNC_MS)) {
    lastMemberSync = now; syncMembers();
  }
  if (now - lastFlush > 15000) { lastFlush = now; flushQueue(); sendHeartbeat(); if (!screenUntil)
idleScreen(); }
}
```

Appendix B: Shared Library Code

These two headers are identical in every project folder (master copies in /common).

IoTCore.h

```
/*
 * IoTCore.h  —  shared runtime for all ESP32 IoT projects in this repo
 * ---------------------------------------------------------------------
 *  - Wi-Fi station with automatic reconnect (credentials stored in flash)
 *  - BLE server using the Nordic UART Service (NUS) layout, so the device
 *    works with the Web Bluetooth dashboard in /docs AND with generic apps
 *    such as "nRF Connect" or "Serial Bluetooth Terminal"
 *  - Line-based command protocol shared by BLE and USB Serial
 *  - Persistent settings (Preferences / NVS): Wi-Fi, cloud keys, PIN, thresholds
 *  - NTP time sync (IST) for timestamps
 *
 *  Wire protocol
 *  -------------
 *  Phone -> ESP32 (write to RX characteristic, text):
 *      auth:1234              unlock control + settings for this connection
 *      status                 device replies with a status JSON line
 *      set:key=value          store a setting (Wi-Fi, cloud key, threshold…)
 *      get:key                read back a setting (secrets are masked)
 *      wifi:SSID,PASSWORD     shortcut for set:ssid + set:pass + reconnect
 *      reboot                 restart the board
 *      <anything else>        forwarded to the project's own command handler
 *  ESP32 -> Phone (notify on TX characteristic):
 *      one JSON object per line, terminated by '\n'
 *      {"type":"data",...}  {"type":"status",...}  {"type":"ack",...}
 *      {"type":"event",...} {"type":"err",...}
 *  Lines longer than one BLE packet are split into chunks; the receiver
 *  joins chunks until it sees '\n'.
 *
 *  Library: NimBLE-Arduino 2.x (lighter than Bluedroid, leaves more RAM for TLS)
 *  Board:   ESP32 Dev Module, Partition Scheme "Minimal SPIFFS (1.9MB APP with OTA)"
 */
#pragma once

#include <Arduino.h>
#include <WiFi.h>
#include <Preferences.h>
#include <NimBLEDevice.h>
#include <ArduinoJson.h>
#include <time.h>

#ifndef DEVICE_PIN
#define DEVICE_PIN "1234"
#endif
#ifndef WIFI_SSID
#define WIFI_SSID ""
#endif
#ifndef WIFI_PASS
#define WIFI_PASS ""
#endif

namespace iot {

// Nordic UART Service UUIDs (widely supported by BLE terminal apps)
static const char* NUS_SERVICE = "6E400001-B5A3-F393-E0A9-E50E24DCCA9E";
static const char* NUS_RX      = "6E400002-B5A3-F393-E0A9-E50E24DCCA9E";  // phone writes here
static const char* NUS_TX      = "6E400003-B5A3-F393-E0A9-E50E24DCCA9E";  // ESP32 notifies here
```

```
// Project command handler: return true if the command was recognised.
using CommandHandler = bool (*)(const String& cmd, const String& arg, bool authed);

// ---------------------------------------------------------------- state
struct CmdMsg { char text[256]; bool fromBle; };

static Preferences        prefs;
static NimBLEServer*      bleServer   = nullptr;
static NimBLECharacteristic* txChr    = nullptr;
static QueueHandle_t      cmdQueue    = nullptr;
static CommandHandler     userHandler = nullptr;
static String             deviceName;
static String             projectId;
static volatile uint16_t  bleMtu      = 23;
static volatile bool      bleAuthed   = false;   // reset on every new connection
static bool               serialAuthed = true;   // USB = physical access, trusted
static bool               timeSynced  = false;
static uint32_t           lastWifiTry = 0;
static String             rxLine;                // assembles multi-packet writes

// ---------------------------------------------------------------- settings
inline String setting(const char* key, const char* def = "") {
  return prefs.isKey(key) ? prefs.getString(key, def) : String(def);
}
inline void saveSetting(const char* key, const String& v) { prefs.putString(key, v); }
inline float settingF(const char* key, float def) {
  String s = setting(key, "");
  return s.length() ? s.toFloat() : def;
}

// ---------------------------------------------------------------- BLE
inline bool bleConnected() { return bleServer && bleServer->getConnectedCount() > 0; }

// Send one line (adds '\n'), split to fit the negotiated MTU.
inline void bleSend(const String& line) {
  if (!bleConnected() || !txChr) return;
  String msg = line + "\n";
  size_t chunk = (bleMtu > 23 ? bleMtu : 23) - 3;
  for (size_t i = 0; i < msg.length(); i += chunk) {
    size_t n = min(chunk, msg.length() - i);
    txChr->setValue((const uint8_t*)msg.c_str() + i, n);
    txChr->notify();
    if (msg.length() > chunk) delay(8);   // give the stack time between chunks
  }
}

// Send to BLE and mirror to USB Serial.
inline void emit(const String& line) {
  Serial.println(line);
  bleSend(line);
}

inline void emitAck(const String& cmd, bool ok, const String& msg = "") {
  JsonDocument d;
  d["type"] = ok ? "ack" : "err";
  d["cmd"] = cmd;
  if (msg.length()) d["msg"] = msg;
  String s; serializeJson(d, s); emit(s);
}

inline void enqueue(const String& s, bool fromBle) {
  if (!cmdQueue || s.length() == 0) return;
```

```
  CmdMsg m; memset(&m, 0, sizeof(m));
  strncpy(m.text, s.c_str(), sizeof(m.text) - 1);
  m.fromBle = fromBle;
  xQueueSend(cmdQueue, &m, 0);
}

class ServerCB : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer* s, NimBLEConnInfo& info) override {
    bleAuthed = false;
    rxLine = "";
    Serial.printf("[BLE] connected %s\n", info.getAddress().toString().c_str());
  }
  void onDisconnect(NimBLEServer* s, NimBLEConnInfo& info, int reason) override {
    bleAuthed = false;
    bleMtu = 23;
    Serial.printf("[BLE] disconnected (reason %d), advertising again\n", reason);
    // NimBLE restarts advertising automatically (advertiseOnDisconnect = true)
  }
  void onMTUChange(uint16_t mtu, NimBLEConnInfo& info) override {
    bleMtu = mtu;
    Serial.printf("[BLE] MTU %u\n", mtu);
  }
};

class RxCB : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* c, NimBLEConnInfo& info) override {
    std::string v = c->getValue();
    // Accept either a single write without newline, or several writes
    // that together form lines terminated by '\n'.
    for (char ch : v) {
      if (ch == '\r') continue;
      if (ch == '\n') { enqueue(rxLine, true); rxLine = ""; }
      else if (rxLine.length() < 250) rxLine += ch;
    }
    if (v.find('\n') == std::string::npos && rxLine.length() && rxLine.length() == v.length()) {
      enqueue(rxLine, true);  // plain one-shot write (e.g. typed in nRF Connect)
      rxLine = "";
    }
  }
};

inline void bleBegin() {
  uint8_t mac[6]; WiFi.macAddress(mac);
  char suffix[6]; snprintf(suffix, sizeof(suffix), "%02X%02X", mac[4], mac[5]);
  deviceName = "IOT-" + projectId + "-" + suffix;

  NimBLEDevice::init(deviceName.c_str());
  NimBLEDevice::setMTU(247);
  bleServer = NimBLEDevice::createServer();
  bleServer->setCallbacks(new ServerCB());

  NimBLEService* svc = bleServer->createService(NUS_SERVICE);
  txChr = svc->createCharacteristic(NUS_TX, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ);
  NimBLECharacteristic* rx = svc->createCharacteristic(
      NUS_RX, NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR, 512);
  rx->setCallbacks(new RxCB());
  // (NimBLE 2.x starts services automatically when advertising begins)

  // 128-bit UUID goes in the advertisement, the name in the scan response
  // (both together would not fit in one 31-byte packet).
  NimBLEAdvertisementData adv, scan;
  adv.setFlags(BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP);
  adv.setCompleteServices(NimBLEUUID(NUS_SERVICE));
```

```
  scan.setName(deviceName.c_str());
  NimBLEAdvertising* a = NimBLEDevice::getAdvertising();
  a->setAdvertisementData(adv);
  a->setScanResponseData(scan);
  a->start();
  Serial.printf("[BLE] advertising as %s\n", deviceName.c_str());
}

// ---------------------------------------------------------------- Wi-Fi
inline bool wifiOK() { return WiFi.status() == WL_CONNECTED; }

inline void wifiConnect() {
  String ssid = setting("ssid", WIFI_SSID);
  String pass = setting("pass", WIFI_PASS);
  if (ssid.isEmpty()) {
    Serial.println("[WiFi] no SSID yet - send  wifi:SSID,PASSWORD  over BLE");
    return;
  }
  Serial.printf("[WiFi] connecting to %s\n", ssid.c_str());
  WiFi.disconnect();
  WiFi.begin(ssid.c_str(), pass.c_str());
  lastWifiTry = millis();
}

inline void wifiLoop() {
  static bool wasUp = false;
  bool up = wifiOK();
  if (up && !wasUp) {
    Serial.printf("[WiFi] connected, IP %s, RSSI %d\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
    configTzTime("IST-5:30", "pool.ntp.org", "time.google.com");
  }
  if (!up && millis() - lastWifiTry > 20000) wifiConnect();   // retry every 20 s
  wasUp = up;
  if (up && !timeSynced && time(nullptr) > 1700000000) {
    timeSynced = true;
    Serial.println("[NTP] time synced");
  }
}

// ISO-8601 local time, or uptime if NTP has not synced yet.
inline String timestamp() {
  time_t now = time(nullptr);
  if (now < 1700000000) return "uptime+" + String(millis() / 1000) + "s";
  struct tm t; localtime_r(&now, &t);
  char buf[32]; strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S+05:30", &t);
  return String(buf);
}

// ------------------------------------------------------- built-in commands
inline bool isSecretKey(const String& k) {
  return k == "pass" || k.endsWith("key") || k == "pin";
}

inline void sendStatus() {
  JsonDocument d;
  d["type"] = "status";
  d["dev"] = deviceName;
  d["wifi"] = wifiOK();
  d["ssid"] = setting("ssid", WIFI_SSID);
  d["rssi"] = wifiOK() ? WiFi.RSSI() : 0;
  d["ip"] = wifiOK() ? WiFi.localIP().toString() : "";
  d["auth"] = bleAuthed;
```

```
  d["heap"] = ESP.getFreeHeap();
  d["up"] = millis() / 1000;
  d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

inline void handleLine(const String& raw, bool fromBle) {
  String line = raw; line.trim();
  if (line.isEmpty()) return;
  int c = line.indexOf(':');
  String cmd = (c < 0 ? line : line.substring(0, c));
  String arg = (c < 0 ? "" : line.substring(c + 1));
  cmd.trim(); cmd.toLowerCase();
  bool authed = fromBle ? bleAuthed : serialAuthed;

  if (cmd == "auth") {
    bool ok = arg == setting("pin", DEVICE_PIN);
    if (fromBle) bleAuthed = ok;
    emitAck(cmd, ok, ok ? "unlocked" : "wrong PIN");
    return;
  }
  if (cmd == "status") { sendStatus(); return; }
  if (cmd == "ping")   { emitAck(cmd, true, "pong"); return; }

  // Everything below changes the device, so it needs the PIN over BLE.
  if (cmd == "set" || cmd == "wifi" || cmd == "reboot" || cmd == "get" || cmd == "forget") {
    if (!authed) { emitAck(cmd, false, "send auth:<PIN> first"); return; }
    if (cmd == "set") {
      int e = arg.indexOf('=');
      if (e <= 0) { emitAck(cmd, false, "use set:key=value"); return; }
      String k = arg.substring(0, e); k.trim();
      String v = arg.substring(e + 1); v.trim();
      if (k.length() > 15) { emitAck(cmd, false, "key too long"); return; }
      saveSetting(k.c_str(), v);
      emitAck(cmd, true, k + " saved");
      if (k == "ssid" || k == "pass") wifiConnect();
      // let the project react to changed thresholds etc.
      if (userHandler) userHandler("setting", k, true);
      return;
    }
    if (cmd == "get") {
      String v = setting(arg.c_str(), "");
      if (isSecretKey(arg) && v.length()) v = "****" + v.substring(max(0, (int)v.length() - 3));
      emitAck(cmd, true, arg + "=" + v);
      return;
    }
    if (cmd == "wifi") {
      int k = arg.indexOf(',');
      String ssid = k < 0 ? arg : arg.substring(0, k);
      String pass = k < 0 ? "" : arg.substring(k + 1);
      saveSetting("ssid", ssid); saveSetting("pass", pass);
      emitAck(cmd, true, "connecting to " + ssid);
      wifiConnect();
      return;
    }
    if (cmd == "forget") { prefs.clear(); emitAck(cmd, true, "all settings erased"); return; }
    if (cmd == "reboot") { emitAck(cmd, true, "rebooting"); delay(300); ESP.restart(); }
  }

  if (userHandler && userHandler(cmd, arg, authed)) return;
  emitAck(cmd, false, "unknown command");
}
```

```
// ------------------------------------------------------------ entry points
// Call once from setup().  id = short project id, e.g. "P01".
inline void begin(const char* id, CommandHandler handler) {
  Serial.begin(115200);
  delay(200);
  projectId = id;
  userHandler = handler;
  prefs.begin("iot", false);
  cmdQueue = xQueueCreate(8, sizeof(CmdMsg));
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  // NOTE: Wi-Fi modem sleep must stay ON while BLE is running (ESP32 coexistence).
  bleBegin();
  wifiConnect();
  Serial.println("[CORE] ready - type 'status' here or connect over BLE");
}

// Call every loop() iteration.
inline void loop() {
  wifiLoop();
  // USB serial commands (handy while developing)
  static String serialLine;
  while (Serial.available()) {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r') { if (serialLine.length()) enqueue(serialLine, false); serialLine =
""; }
    else if (serialLine.length() < 250) serialLine += ch;
  }
  CmdMsg m;
  while (cmdQueue && xQueueReceive(cmdQueue, &m, 0) == pdTRUE) handleLine(String(m.text), m.fromBle);
}

// Round for compact JSON. NaN stays NaN, which ArduinoJson writes as null.
inline float rnd(float v, int dp = 1) {
  if (isnan(v)) return v;
  float m = powf(10, dp);
  return roundf(v * m) / m;
}

// Helper: build and send a {"type":"data",...} line from a filled document.
inline void sendData(JsonDocument& d) {
  d["type"] = "data";
  String s; serializeJson(d, s); emit(s);
}

inline void sendEvent(const String& name, const String& msg) {
  JsonDocument d;
  d["type"] = "event"; d["name"] = name; d["msg"] = msg; d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

}  // namespace iot
```

Cloud.h

```
/*
 * Cloud.h  —  free-tier cloud helpers used by the projects
 * ------------------------------------------------------------
 *  ThingSpeak   : HTTPS REST "update" (free tier: 1 update / 15 s per channel)
 *  Adafruit IO  : MQTT over TLS (free tier: 30 data points / minute, 10 feeds)
 *  Supabase     : PostgREST over HTTPS (free tier: 500 MB Postgres)
 *
```

```
 *  Keys are read from settings (stored over BLE with set:key=value) so no
 *  secret has to be hard-coded or committed to GitHub:
 *     tskey    ThingSpeak Write API Key
 *     aiouser  Adafruit IO username        aiokey  Adafruit IO key
 *     sburl    https://<ref>.supabase.co   sbkey   Supabase publishable/anon key
 *
 *  TLS note: certificate checking is disabled with setInsecure() to keep the
 *  projects beginner-friendly and independent of CA rotations. Traffic is
 *  still encrypted. For production, pin the root CA with setCACert().
 */
#pragma once

#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <PubSubClient.h>
#include "IoTCore.h"

// Optional compile-time defaults (normally left empty and set over BLE)
#ifndef TS_WRITE_KEY
#define TS_WRITE_KEY ""
#endif
#ifndef AIO_USERNAME
#define AIO_USERNAME ""
#endif
#ifndef AIO_KEY
#define AIO_KEY ""
#endif
#ifndef SUPABASE_URL
#define SUPABASE_URL ""
#endif
#ifndef SUPABASE_KEY
#define SUPABASE_KEY ""
#endif

namespace cloud {

// ================================================================ ThingSpeak
// vals[i] -> field(i+1). Pass NAN to skip a field. Returns true on success.
inline bool thingSpeakUpdate(const float* vals, int n, const String& status = "") {
  if (!iot::wifiOK()) return false;
  String key = iot::setting("tskey", TS_WRITE_KEY);
  if (key.isEmpty()) { Serial.println("[TS] no tskey set"); return false; }
  String url = "https://api.thingspeak.com/update?api_key=" + key;
  for (int i = 0; i < n && i < 8; i++)
    if (!isnan(vals[i])) url += "&field" + String(i + 1) + "=" + String(vals[i], 2);
  if (status.length()) {
    String st = status; st.replace(" ", "%20");
    url += "&status=" + st;
  }
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, url)) return false;
  int code = http.GET();
  String body = http.getString();
  http.end();
  bool ok = code == 200 && body.toInt() > 0;   // ThingSpeak returns the entry id, 0 = rejected
  Serial.printf("[TS] HTTP %d entry=%s\n", code, body.c_str());
  return ok;
}

// ================================================================ Supabase
// Generic PostgREST call. method: GET/POST/PATCH. path e.g. "/rest/v1/readings"
inline int supabaseRequest(const char* method, const String& path, const String& body,
```

```
                          String* response = nullptr, const char* prefer = "return=minimal") {
  if (!iot::wifiOK()) return -1;
  String base = iot::setting("sburl", SUPABASE_URL);
  String key  = iot::setting("sbkey", SUPABASE_KEY);
  if (base.isEmpty() || key.isEmpty()) { Serial.println("[SB] sburl/sbkey not set"); return -2; }
  if (base.endsWith("/")) base.remove(base.length() - 1);
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, base + path)) return -3;
  http.addHeader("apikey", key);
  http.addHeader("Authorization", "Bearer " + key);
  http.addHeader("Content-Type", "application/json");
  if (prefer) http.addHeader("Prefer", prefer);
  int code = http.sendRequest(method, body);
  if (response) *response = http.getString();
  else if (code >= 300) Serial.printf("[SB] %s\n", http.getString().c_str());
  http.end();
  Serial.printf("[SB] %s %s -> %d\n", method, path.c_str(), code);
  return code;
}

inline bool supabaseInsert(const char* table, const String& json) {
  int c = supabaseRequest("POST", String("/rest/v1/") + table, json);
  return c >= 200 && c < 300;
}

// filter is a PostgREST filter, e.g. "id=eq.1"
inline bool supabaseUpdate(const char* table, const String& filter, const String& json) {
  int c = supabaseRequest("PATCH", String("/rest/v1/") + table + "?" + filter, json);
  return c >= 200 && c < 300;
}

// query e.g. "select=*&id=eq.1"  -> JSON array text in out
inline bool supabaseSelect(const char* table, const String& query, String& out) {
  int c = supabaseRequest("GET", String("/rest/v1/") + table + "?" + query, "", &out, nullptr);
  return c == 200;
}

// ============================================================ Adafruit IO
// MQTT wrapper with a simple rate limiter to respect the 30 msg/min limit.
using FeedCallback = void (*)(const String& feed, const String& value);

class AdafruitIO {
 public:
  // requestLast = true asks Adafruit IO for each feed's last value on connect
  // (useful for switches changed while offline; unsafe for things like pumps)
  void begin(FeedCallback cb, const char* const* subFeeds, int nSub, bool requestLast = true) {
    cb_ = cb; subs_ = subFeeds; nSub_ = nSub; requestLast_ = requestLast;
    tls_.setInsecure();
    mqtt_.setClient(tls_);
    mqtt_.setServer("io.adafruit.com", 8883);
    mqtt_.setBufferSize(512);
    mqtt_.setKeepAlive(60);
    mqtt_.setCallback([](char* topic, byte* payload, unsigned int len) {
      String t(topic), v;
      for (unsigned int i = 0; i < len; i++) v += (char)payload[i];
      int p = t.lastIndexOf('/');
      String feed = t.substring(p + 1);
      if (instance()->cb_) instance()->cb_(feed, v);
    });
    instance() = this;
  }
```

```
  void loop() {
    if (!iot::wifiOK()) return;
    if (!mqtt_.connected()) {
      if (millis() - lastTry_ < 10000) return;
      lastTry_ = millis();
      user_ = iot::setting("aiouser", AIO_USERNAME);
      String key = iot::setting("aiokey", AIO_KEY);
      if (user_.isEmpty() || key.isEmpty()) { Serial.println("[AIO] aiouser/aiokey not set"); return; }
      String id = "esp32-" + String((uint32_t)ESP.getEfuseMac(), HEX);
      Serial.println("[AIO] connecting...");
      if (mqtt_.connect(id.c_str(), user_.c_str(), key.c_str())) {
        Serial.println("[AIO] connected");
        for (int i = 0; i < nSub_; i++) {
          mqtt_.subscribe(topic(subs_[i]).c_str());
          if (requestLast_) mqtt_.publish((topic(subs_[i]) + "/get").c_str(), "");
        }
        // subscribe to throttle/error topics so we learn about rate limits
        mqtt_.subscribe((user_ + "/throttle").c_str());
      } else {
        Serial.printf("[AIO] failed, state %d\n", mqtt_.state());
      }
    }
    mqtt_.loop();
  }

  bool connected() { return mqtt_.connected(); }

  // Returns false if not connected or if the per-minute budget is used up.
  bool publish(const char* feed, const String& value) {
    if (!mqtt_.connected()) return false;
    uint32_t now = millis();
    if (now - windowStart_ > 60000) { windowStart_ = now; sent_ = 0; }
    if (sent_ >= 28) { Serial.println("[AIO] rate budget used, skipping"); return false; }
    sent_++;
    return mqtt_.publish(topic(feed).c_str(), value.c_str());
  }
  bool publish(const char* feed, float v, int dp = 2) { return publish(feed, String(v, dp)); }

 private:
  String topic(const char* feed) { return user_ + "/feeds/" + feed; }
  static AdafruitIO*& instance() { static AdafruitIO* p = nullptr; return p; }

  WiFiClientSecure tls_;
  PubSubClient mqtt_;
  FeedCallback cb_ = nullptr;
  const char* const* subs_ = nullptr;
  int nSub_ = 0;
  bool requestLast_ = true;
  String user_;
  uint32_t lastTry_ = 0, windowStart_ = 0;
  int sent_ = 0;
};

}  // namespace cloud
```